Memory Management Mechanism — AgenticSys_v2

Snapshot: 2026-05-11. Reflects the codebase after commits `8ddce4e..fecdd3e` (Phase 1–3 of the Memory-Management PRD, plus async distiller and chart-to-trace refinements).

---

1. Why this exists — the problem being solved

The case-review system slows down across long reviewer conversations because the orchestrator's `input_history` accumulates every turn's full specialist tool outputs (5–50 KB each), feeding 100–500 KB of context back to the model by turn 8–10. Two compounding deficiencies:

1. **No semantic recall across turns.** Specialists had only intra-orchestrator-run memory, so they re-ran identical `summarize_trend` / `aggregate_column` queries on every follow-up.
2. **Quantitative findings only as prose.** Reviewers find numeric trends and concentrations easier to parse as charts than narrative numbers.

The memory-management subsystem replaces unbounded raw-context replay with **structured, semantically-targeted recall**, paired with a routing signal so the orchestrator favors warm specialists on follow-ups.

---

2. The core idea (one paragraph)

Each specialist's findings are **distilled** into atomic, quantitative `KnowledgePoints` by a second-pass agent. These KPs are stored per-specialist in a session-scoped `specialist_kb`. On the specialist's next call, only the **active set** (latest per topic) is fed back as a short digest preface — so the specialist sees what it already knows without replaying the raw tool output. The orchestrator's `input_history` is pruned in parallel (old turns' tool-result payloads replaced by stubs), and a one-line **warmth hint** prepended to the user question tells the orchestrator which specialists are already loaded with knowledge for this case.

---

3. The four memory layers

The system maintains four distinct memory layers running in parallel, all scoped to a `CaseSession` and surviving across reviewer turns within that session.

```
┌──────────────────────────────────────────────────────────────────────┐
│  CaseSession  (server.py:67)                                         │
│                                                                      │
│   input_history ──── pruned every turn (keep last 2 turns intact)   │
│   specialist_kb ─── per-specialist list[KnowledgePoint], append-only│
│   qa_cache ──────── exact + near-duplicate Q→A cache                 │
│   (warmth hint) ── derived from specialist_kb, prepended each turn   │
└──────────────────────────────────────────────────────────────────────┘
                              │
                              ▼ per turn, threaded as AppContext
┌──────────────────────────────────────────────────────────────────────┐
│  AppContext  (agent_factories/app_context.py)                        │
│                                                                      │
│   _specialist_kb       → SAME DICT BY REFERENCE as session's KB      │
│   _specialist_histories → per-specialist intra-turn chat history    │
```

```
│   _specialist_call_cache→ per-AppContext (name, sub_q) dedup        │
│   _distiller           → the shared second-pass agent               │
│   _pending_distillers  → fire-and-forget task handles               │
│   _turn_id             → tagged onto each new KP                    │
└─────────────────────────────────────────────────────────────────────┘
```

3.1 Orchestrator `input_history` — bounded conversational memory

- Field: `CaseSession.input_history` (`server.py:80`).
- Behavior: each turn appends user message + tool calls + tool outputs + final answer; replayed verbatim into `Runner.run_streamed` on the next turn.
- Bounding: `_prune_input_history` (`server.py:238`) runs after each turn.
  - Identifies turn boundaries by `{"role": "user", ...}` entries.
  - Keeps the last **2 reviewer turns** intact (`_INPUT_HISTORY_KEEP_RECENT_TURNS`).
  - In older turns, replaces `function_call_output.output` with the stub: `> "(elided — earlier-turn specialist output; see the specialist's KB digest, which is prepended to each new sub-question.)"`
  - Function-call records themselves are preserved □ orchestrator still sees *that* a tool ran, just not the heavy payload.
- Authoritative replay path for elided findings is the KB digest, **not** the history.

3.2 Per-specialist Knowledge Base — cross-turn semantic memory

- Field: `CaseSession.specialist_kb: dict[str, list[KnowledgePoint dict]]` (`server.py:97`).
- Shared by reference into each turn's `AppContext._specialist_kb` so the redacting_tool wrapper's writes persist back to the session automatically.

`KnowledgePoint` schema (`models/types.py:58`):

field	type	purpose
`topic`	`str` (snake_case slug)	grouping key for supersession (latest per topic wins)
`claim`	`str` (1 sentence)	numbers + named entities + time window; faithfully extracted
`numbers`	`list[dict]`	data series behind the claim (trends, breakdowns, breaches)
`viz`	`dict | None`	optional `{kind, x_field, y_fields}` chart spec
`source_call`	`str`	the tool invocation that produced the data
`captured_at_turn`	`str | None`	short hex turn id — chronological audit
`confidence`	`high|medium|low`	reflects the specialist's hedging

Supersession model:

- The list is **append-only** — never mutated, never deleted (audit trail).
- `_active_kps` in `redacting_tool.py:36` iterates and keeps the **last-seen entry per topic**; this is what the specialist sees on its next call.
- Older entries with the same topic stay in the list so the logger can reconstruct what was believed at any point in the session.

Read path (`redacting_tool.py:294`):

- Only on the **first call** to a specialist within a turn, `_format_kb_digest` prepends a one-line-per-active-KP preface:

```
[YOUR KNOWLEDGE BASE — facts established earlier this session.
 Refer to these BEFORE re-running queries; only re-query when the new
 question goes beyond what's recorded here, or when a value needs verification.]

- **monthly_spend_trend** [high]: Spend rose from $300 (2024-11) to $1,100 (2025-
03)…  _via `summarize_trend('spends',…)`_
- **top_merchants_by_sum** [medium]: S BERTRAM accounts for 38% of recurring spend ($642K of $
```

- Intra-turn follow-ups inherit the digest via the per-specialist conversation history (`_specialist_histories`), so re-prepending would duplicate.

Lifecycle: cleared by `/rewind` alongside `input_history` and `qa_cache` so a session reset wipes everything.

3.3 Async distiller — the KB writer

- Agent: `agent_factories/distiller_agent.py` — stateless, structured output (`DistillerOutput` □ `knowledge_points[]`), one shared instance per orchestrator.
- Built once in `Orchestrator.__init__` (`orchestrator/orchestrator.py:61`) and exposed on `AppContext._distiller`.

Why a separate second pass instead of inline emission:

- Distillation is a different cognitive task than analysis. Asking the specialist to do both reliably bloats its prompt and degrades both.
- A narrowly-scoped agent with a strict output schema is more faithful (less paraphrasing) and cheaper to iterate.
- Failures degrade gracefully to "no KB update this turn" — the specialist's answer is unaffected.

Strict extraction rules (from the distiller prompt):

- Faithful extraction only — every claim grounded directly in the SpecialistOutput; hedges preserved.
- Atomic — one quantitative fact per point; a 12-month trend series is ONE point, not 12.
- Quantitative bias — prefer numbers, named entities, comparisons; skip pure-narrative claims.
- Skip data-absence (already in SpecialistOutput's `data_gaps`).

Fire-and-forget scheduling (`redacting_tool.py:385`):

After a specialist successfully returns:

```python
task = asyncio.create_task(
    _distill_and_persist(app_ctx, name, redacted_in, result.final_output),
    name=f"distill-{name}",
)
app_ctx._pending_distillers.append(task)
return payload   # orchestrator gets answer immediately
```

- Orchestrator receives the specialist's payload **without** paying the distiller round-trip on the critical path.
- Server.py awaits all pending tasks at **end-of-turn** (`server.py:1014`, 60s budget) so the KB is fully populated before the next turn's warmth digest is built.
- Per-distiller timeout: 30s (`_DISTILLER_TIMEOUT_S`). On timeout / error, logged as `distiller_failed`; the specialist answer is unaffected.

Chart rendering side-effect:

When a distilled KP carries `viz` + non-empty `numbers`, `_distill_and_persist` also:

- Calls `kp_to_vega_spec(kp_dict)` and stores the Vega-Lite v5 spec on the KP (`kp_dict["vega_spec"] = spec`).

- Calls `render_chart(kp_dict, charts_dir, turn_id=…)` to write `reports/<case_id>/charts/<turn_id>-<topic>.png`; stores the relative URL on `kp_dict["image_path"]`.
- Failures (matplotlib error, missing fields) are logged but never raised — KP still lands in the KB, just without a chart.

Charts surface in the reasoning-trace SSE panel (not inline in the chat answer) via `_collect_turn_charts` (`server.py:323`).

3.4 Warmth hint — routing signal for the orchestrator

- Built by `_format_kb_warmth_hint` (`server.py:298`).
- Format: `[KB-warmth: spend_payments (3 KPs), modeling (5 KPs). Strongly consider reusing warm specialists for in-domain follow-ups.]`
- Sorted by descending KP count; specialists with zero KPs are omitted (no "(0 KPs)" noise).
- Prepended to the redacted user question on every turn after the first (`server.py:684`):

```
framed_question = f"{warmth_hint}\n\n{verdict.redacted_question}"
```

- The `team_construction` skill is updated to treat this as a primary follow-up routing signal: "favor reusing warm specialists unless the question's domain has clearly shifted."
- **No hard skip** of team construction — the orchestrator retains full LLM judgment; the hint is *a signal*, not a programmatic bypass.
- Logged as `kb_warmth_hint_emitted` with `turn_id`, `warm_specialists`, `hint_length` (no PII, structural counts only).

---

4. Supporting caches

4.1 Per-specialist intra-turn history

- `AppContext._specialist_histories: dict[str, list]` (`app_context.py:22`).
- Updated by `redacting_tool` after each specialist run with `result.to_input_list()`.
- Lets a follow-up tool call to the same specialist **within the same AppContext** see what was already asked / answered, instead of starting fresh.
- Resets per-AppContext (i.e. per-turn).

4.2 Per-specialist call dedup

- `AppContext._specialist_call_cache: dict[tuple[name, normalized_subq], str]` (`redacting_tool.py:270`).
- Same `(specialist, normalized_sub_question)` within the same context returns the cached payload rather than re-running.
- Caps cost when the orchestrator (especially in safechain mode where parallel-tool-call semantics aren't native) emits the same call multiple times in one turn with trivial wording variations.
- `_normalize_subq` collapses whitespace + lowercases.

4.3 Session QA cache

- `CaseSession.qa_cache: dict` (`server.py:89`).
- Keyed by `_normalize_q(redacted_question)`; value carries the cached FinalAnswer fields.
- **Exact-match** on the redacted reviewer question □ skips the orchestrator entirely on repeats.

- **Near-duplicate** path (`server.py:589`): `ScreenVerdict.near_duplicate_of` (computed by the relevance_check skill on subject + time-range + scope) re-keys into the cache for fuzzy matches.
- Logged as `qa_cache_hit` / `qa_cache_hit_near_duplicate`.

---

5. End-to-end flow per turn

```
┌──────────────────────────────────────────────────────────────────────┐
│ 1. Server receives user question                                     │
│    • screen + redact via ChatAgent                                   │
│    • near-duplicate check against qa_cache → replay if hit           │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 2. Build framed question                                             │
│    • _format_kb_warmth_hint(sess.specialist_kb)                      │
│    • framed = f"{warmth_hint}\n\n{redacted_question}"                │
│    • run_input = sess.input_history + [{"role":"user", framed}]      │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 3. Orchestrator routes; specialists called via redacting_tool        │
│    For each specialist call:                                         │
│      a. dedup-cache hit? → return cached payload, done               │
│      b. first call this turn? → prepend KB digest (active KPs)       │
│      c. prior intra-turn history? → prepend that instead             │
│      d. Runner.run(inner, run_input, max_turns=25, timeout=240s)     │
│      e. redact_payload(result.final_output) → orchestrator           │
│      f. schedule fire-and-forget distiller task                      │
│      g. save updated history to _specialist_histories                │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 4. Orchestrator emits FinalAnswer                                    │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 5. End-of-turn server bookkeeping                                    │
│    • drain _pending_distillers (60s budget)                          │
│      → KB now reflects this turn's KPs (incl. PNG + vega_spec)       │
│    • _collect_turn_charts → emit `chart` SSE events                  │
│    • _prune_input_history → stub old tool outputs                    │
│    • write qa_cache entry keyed on this question                     │
└──────────────────────────────────────────────────────────────────────┘
```

---

6. Design trade-offs (the "why" behind the shape)

decision	rationale
Second-pass distillation instead of inline KP emission	Specialist + distiller are different cognitive tasks; combining them bloats prompts and degrades faithfulness. Separate agent is also cheaper to iterate.
Fire-and-forget distiller	The orchestrator should not pay the distiller round-trip on the critical path. The KB only needs to be fresh for the *next* turn, so end-of-turn drain is sufficient.
Append-only KB with implicit supersession	Audit trail is preserved (operators can reconstruct what was believed at any turn) while the active view stays small.
Short digest preface, not full KB replay	The job is to *prevent re-querying*, not to replay every detail. The specialist can still re-query when verification is needed.
Warmth hint as soft signal, not hard short-circuit	Keeps team construction LLM-driven so the orchestrator can override on clear domain shifts. Programmatic bypass would be a separate exact-near-duplicate fast-path.
`input_history` pruning preserves call records	Orchestrator still sees *that* a tool was invoked (avoids spurious re-calls), but elides the heavy payload (saves tokens).
Charts to reasoning-trace panel, not chat answer	Keeps the chat clean (text only); reviewers get click-to-open access tied to the specific finding. Vega-Lite spec is preserved for future interactive UIs.
No RAG over KB	Digest is passed verbatim; embedding-based retrieval is deferred until KBs grow past ~50 entries per specialist (not observed in current sessions).
Domain specialists see only their own KB	Cross-domain sharing (e.g. through `general_specialist`) is intentionally deferred until use cases firm up.

---

7. Observability hooks

The EventLogger emits the following memory-related events:

event	trigger	payload (structural only — no PII)
`kb_warmth_hint_emitted`	non-empty hint built before a turn	`turn_id`, `warm_specialists[{name, n_kps}]`, `hint_length`
`distiller_kps_added`	distiller produced ≥ 1 KP	`specialist`, `n_added`, `kb_size_now`, `topics[]`, `n_with_charts`
`distiller_failed`	distiller run errored or timed out	`specialist`, `error_type`, `error_message` (truncated)
`distiller_drain_timeout`	end-of-turn drain exceeded 60s	`turn_id`, `n_pending`
`distiller_outer_failure`	task creation itself failed (rare)	`specialist`, `error_type`, `error_message`
`input_history_pruned`	history pruning produced any elisions	counts of items elided, bytes saved
`specialist_call_deduped`	dedup cache served a call	`specialist`, `sub_question_norm`
`qa_cache_hit` / `qa_cache_hit_near_duplicate`	session-level QA cache served the answer	`turn_id`, `cached_question` (redacted)
`qa_cache_store`	new answer cached after a turn	structural counts

---

8. Known limits / explicit non-goals

- **No interactive client-side charts** — Phase 2 ships static PNG + Vega-Lite spec; interactive UI is future work.
- **No hard-skip routing** — Phase 3 deliberately keeps team construction as an LLM decision.
- **No RAG / embedding retrieval over KB** — deferred until KBs grow past ~50 entries.
- **No cross-specialist KB sharing** (except future `general_specialist`).
- **No chart re-rendering for historical KPs** — only KPs captured in the current turn render charts; pre-Phase-2 KPs aren't back-filled.
- **No "reset case" admin path** — `/rewind` wipes session state but chart files persist on disk for audit.

---

9. Files of interest

concern	file
Session state, warmth, pruning	`server.py` (`CaseSession`, `_prune_input_history`, `_format_kb_warmth_hint`)
KB read path + distiller scheduling	`agent_factories/redacting_tool.py`
Per-turn context plumbing	`agent_factories/app_context.py`
Distiller agent + prompt	`agent_factories/distiller_agent.py`
`KnowledgePoint` schema	`models/types.py`
Distiller construction	`orchestrator/orchestrator.py`
Chart rendering	`tools/viz_renderer.py`
Routing skill (consumes warmth)	`skills/workflow/team_construction.md`
Original PRD	`tasks/prd-memory-management.md`